The Causal Instability Problem

An Empirical Audit of 23 Datasets and 4 Generators

Tim Menzies *et al.*

prof, cs, **ncstate**, usa
http://timm.fyi

Mar 2025

Motivation: The Allure of the "Why"

Claim: Software Engineering (SE) is tired of correlations; we want **causes**.

- We don't want to know that "Size correlates with Defects."
- We want to know: "If I change this module, will I stop the bugs?"
- Causal Discovery Algorithms (CDAs) promise to find these hidden structures automatically.

But as the "just throw an LLM at it" phase ends, so must the "just throw a CDA at it" phase. [cite: 3]

The Reality: The Instability Problem

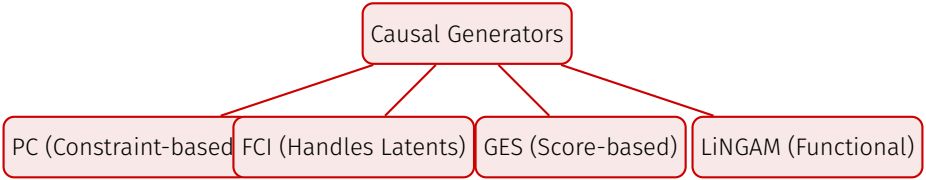
Claim: Automatic causal graphs are currently too unstable to trust for industrial SE.

The Definition of Instability

Small changes to the input data result in **large, unpredictable changes** in the resulting graph.

- If the graph changes every time you re-run it, it isn't an "explanation."
- It is just noise masquerading as a diagram.

We audited the four most prominent paradigms in causal discovery. They all share the same "Unfair Advantage" of automation—and the same flaws in stability. [cite: 7]



The Archive: 23 Datasets across 30 Years

- We didn't test on toy data. We used the **PROMISE** and **JURECZKO** repositories.
- **Breadth:** Ant, Camel, JEdit, Lucene, POI, Tomcat, Velocity, and more.
- **Depth:** Multiple releases per project (e.g., Ant 1.3, 1.4, 1.5, 1.6, 1.7).

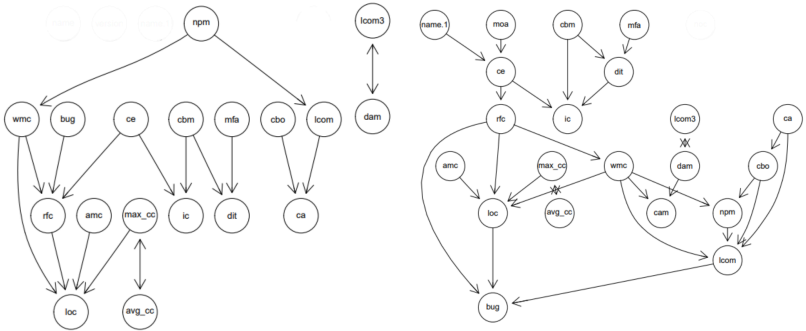
Not a green-field project

This is a audit of how these tools behave on real-world SE evolution. [cite: 11]

The Yardstick: The Jaccard Index

- $J(A, B) = \frac{|E_A \cap E_B|}{|E_A \cup E_B|}$
- $J = 1.0$: The graphs are identical.
- $J = 0.0$: The graphs have zero edges in common.
- **Rule:** If J is low, your "Why Engine" is broken. [cite: 25]

We measured Jaccard across versions, across tools, across data subsets. e.g. v5,v7:



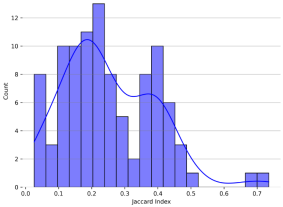
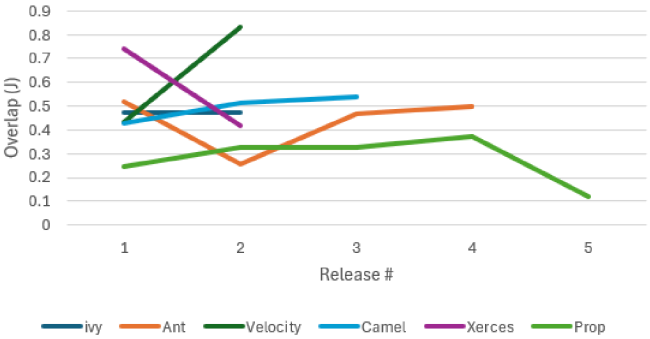


Fig. 2: Jaccard index results seen in 100 experiments where two causal graph generators are applied to the same data set.

Finding 1: The Time Flip

Question: Does the "cause" of defects stay the same from Version 1 to Version 2?

- **The Result:** In nearly all cases, over half the edges vanish between releases.
- Median stability across versions: **Extremely Low**.
- **The Risk:** A manager fixes a "cause" in Version 1.5 that no longer exists in Version 1.7.



Finding 2: The Domain Gap

Question: If I learn the causal graph for Camel, can I use it for Tomcat?

- **The Result:** Over two-thirds of causal connections differ between projects.
- This suggests causal discovery in SE is **highly idiosyncratic**.
- Generalization is a dangerous assumption in this landscape.

Table 6: **RQ2 results:** Inter-Project Jaccard Values. This matrix is symmetrical across the diagonal (so the values of blank cell i, j can be found in cell j, i).

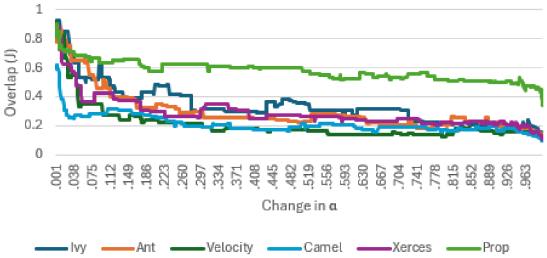
Project	**Ivy**	**Ant**	**Velocity**	**Camel**	**Xerces**
Ivy	1	.28	.37	.25	.26
Ant		1	.28	.19	.20
Velocity			1	.20	.26
Camel				1	.29
Xerces					1

Finding 3: The Fragility of Alpha

Question: Does changing the significance level α (e.g., 0.01 to 0.05) break the graph?

- **The Result:** Minor tuning adjustments generate radically different causal structures.
- These tools are **not robust** to standard hyperparameter variations.

Fig. 4: **RQ3 results:** Graph of Jaccard values when shifting alpha $\Delta = 0.001$. All graphs compared to the graph from $\alpha = 0.001$.



Key Design Warning

If your "Why" depends on whether $\alpha = 0.01$ or 0.05, you don't have a "Why." [cite: 25]

Finding 5: Other Data? Other Causal Generators?

$$\underbrace{\begin{pmatrix} RQ1 \\ RQ2 \\ RQ3 \\ RQ4 \end{pmatrix}}_{4\ research\ questions} \times \underbrace{\begin{pmatrix} PC \\ FCI \\ GES \\ LiNGAM \end{pmatrix}}_{4\ generators} \times \underbrace{\begin{pmatrix} 18\ defect\ prediction \\ 2\ software\ configuration \\ 3\ software\ process\ management \end{pmatrix}}_{3\ tasks,\ 23\ data\ sets}$$

Table 7: Jaccard indexes seen in different experiments.

		Causal graph generator			
data	**question**	**PC**	**FCI**	**GES**	**LiNGAM**
		μ (σ)	μ (σ)	μ (σ)	μ (σ)
defect	**RQ1** : across releases	.67 (.07)	.43 (.11)	.50 (.08)	.39 (.06)
defect	**RQ2** : across projects	.42 (.04)	.19 (.07)	.34 (.04)	.24 (0.5)
defect	**RQ3** : changing α	.51 (.06)	.27 (.07)	N/A	N/A
config	**RQ3** : changing α	.27 (.07)	.10 (.04)	N/A	N/A
process	**RQ3** : changing α	.12 (.03)	.06 (.02)	N/A	N/A
defect	**RQ4** : subsets	.84 (.06)	.59 (.12)	.70 (.07)	.52 (.10)
config	**RQ4** : subsets	.75 (.18)	.59 (.26)	.73 (.16)	.57 (.20)
process	**RQ4** : subsets	.64 (.11)	.54 (.13)	.59 (.09)	.29 (.07)

The Why: Why Do Graphs Fail in SE?

- **Data Sparsity:** SE data is often "fat and short" (many features, few rows). [cite: 9]
- **Noisy Metrics:** Static code features are proxies, not direct physical laws.
- **Stochastic Nature:** Software evolution is driven by human choice, not just mathematical necessity. [cite: 37]

We need **hybrid workflows** that combine these graphs with symbolic checks. [cite: 5, 25]

Roadmap: Moving Beyond the "Hype"

1. **Intersection Filters:** Only report edges found by *at least two* different generators.
2. **Stability Audits:** Always run a "10% drop test" before publishing a graph.
3. **Type-Safe Checks:** Use Z3 to reject causal links that violate known SE constraints. [cite: 14, 25]
4. **Ensemble Causality:** Treat causal discovery as an optimization problem over an ensemble of graphs. [cite: 33]

Why Me, Why Now

- We have the archive: 23 projects over 30 years. [cite: 35]
- We have the machinery: thousands of GPU hours auditing these generators. [cite: 36]
- **The Conclusion:** Causal graphs are a powerful *hypothesis generator*, but a dangerous *decision maker*—until we solve the instability problem.

Closing pitch

This is the moment to connect causal mining with symbolic safety and stochastic rigor. [cite: 38]